DATA STRUCTURES AND ALGORITHMS

UNIT – 4 | DATA MANAGEMENT MINI TOOLKIT (DMMT)

ASSIGNMENT REPORT

Student Details

Name	Omesh Verma
Roll No.	2501730331
Program	B.Tech (CSE AIML)
Section	A
Course Title	Data Structures and Algorithms
Course Code	ETCCDS202
Unit Number	Unit – 4
School	School of Engineering & Technology

1. Objective

- To understand non-linear and advanced data structures.
- To implement Binary Search Tree, Graph, and Hash Table using Python.
- To perform efficient searching, traversal, and data storage operations.
- To understand real-world applications of these data structures.

2. Introduction

Non-linear data structures store data in a hierarchical or network-based manner instead of sequential order. These structures provide efficient ways to store, retrieve, and process large amounts of data. Examples include trees, graphs, and hash tables, which are widely used in databases, networking, and caching systems.

3. Implementation Overview

The following components were implemented in this assignment:

- Binary Search Tree (BST): Supports insertion, search, deletion, and inorder traversal.
- Graph (Adjacency List): Represents a directed weighted graph.

- BFS Traversal: Explores nodes level by level using queue.
- DFS Traversal: Explores nodes depth-wise using recursion.
- Hash Table: Implements key-value storage using separate chaining.

The table below summarizes the real-world applications of each implemented structure:

Structure	Key Feature	Real-World Use
BST	Sorted inorder traversal	Databases, symbol tables
Graph – BFS	Level-by-level exploration	Shortest path, web crawling
Graph – DFS	Deep path exploration	Cycle detection, maze solving
Hash Table	O(1) average lookup	Caching, dictionaries, sets

4. Analysis

Binary Search Tree (BST)

- Inorder traversal prints elements in sorted order.
- Average time complexity is O(log n).
- Worst-case time complexity is O(n) when the tree is skewed.

Graph Traversal

BFS (Breadth First Search)

- Visits nodes level by level.
- Useful for shortest path in unweighted graphs.

DFS (Depth First Search)

- Explores nodes deeply before backtracking.
- Useful for cycle detection and path finding.

Hash Table

- Collision occurs when multiple keys map to the same index.
- Separate chaining resolves collisions using lists.
- Average time complexity is O(1).
- Worst-case time complexity is O(n).

The table below summarizes the time complexity of all implemented structures:

Structure	Operation	Average Case	Worst Case
BST	Search / Insert / Delete	O(log n)	O(n) skewed
Graph (BFS)	Traversal	O(V + E)	O(V + E)
Graph (DFS)	Traversal	O(V + E)	O(V + E)
Hash Table	Search / Insert / Delete	O(1)	O(n) collision

5. Conclusion

This assignment provided a clear understanding of advanced data structures and their applications. It demonstrated how efficient data storage and retrieval can be achieved using BST, graphs, and hash tables. These structures are essential for solving complex real-world problems.

6. References

- Lecture notes and class materials
- Basic Python documentation